Foundations of Computational Linguistics - Lecture 02

Lecture overview

Lecture 02 establishes the hierarchical structure of language across phonology, morphology, syntax, semantics, and pragmatics, detailing how ambiguity at each level complicates computational tasks like speech recognition and machine translation. Key challenges include resolving homophones in phonology, distinguishing word categories in morphology (e.g., "flies" as noun vs. verb), and handling structural attachment in syntax. Machine translation further struggles with divergent word orders, one-to-many lexical mappings, polysemy (e.g., French "avocat"), and subject omission, necessitating context-aware disambiguation rather than literal substitution.

The lecture transitions to sentiment analysis and statistical modeling, introducing the Naive Bayes classifier built on the Bag-of-Words representation and conditional independence assumptions. It covers the mathematical derivation of Bayes' Theorem, the necessity of Laplace smoothing to handle zero probabilities, and the limitations of simple keyword matching when facing negation, sarcasm, or irony. Finally, the session outlines rigorous model evaluation protocols, emphasizing the separation of training, development, and test sets to prevent data leakage, the use of baselines and human upper bounds for context, and the preference for precision, recall, and F1 scores over accuracy in imbalanced or sequence-based tasks like Named Entity Recognition.

Slide 1: Levels of linguistic structure

Linguistic structure is organized into five levels: phonology (sounds), morphology (word structure), syntax (sentence structure), semantics (word meanings and their combination), and pragmatics (utterance meaning within a discourse context). These components collectively define how language functions from individual sounds to contextual communication.

Slide 2: Ambiguity: phonology

Phonology concerns the conceptual basic elements of sound within linguistics, distinct from phonetics which deals with actual articulated sounds. This level introduces ambiguity through homophones, such as "buy," "by," and "bye," which are different words written differently but sounding exactly the same. Resolving this ambiguity is critical for speech recognition systems that must transform a spoken signal into an accurate written signal.

Slide 3: Ambiguity: morphology

Morphological ambiguity arises from homonyms or homographs-words written identically but belonging to different categories, such as "flies" (noun/verb), "past" (preposition/noun), or "like" (preposition/verb). This phenomenon complicates both syntactic and semantic analysis by requiring disambiguation of word categories within context. The issue is highlighted in sentences like "Time flies like an arrow" versus "Fruit flies like a banana," where identical surface forms demand distinct interpretations of "flies" and "like" to determine correct sentence structure and meaning.

Slide 4: Ambiguity: syntax

Structural ambiguity occurs when sentences share identical word-level structures but yield different meanings based on syntactic attachment. In "John hits the ball with a bat," the prepositional phrase modifies the verb, indicating the bat is the instrument of hitting. Conversely, in "John hits the man with a hat," the phrase attaches to the noun "man," describing the man as wearing a hat rather than being hit by one.

Slide 5: Ambiguity: semantics

Semantic ambiguity arises when a single word form carries multiple distinct meanings depending on context. The word "pen" illustrates this by functioning as a writing instrument, an animal enclosure, a metaphor for jail, or a verb meaning to write. Proper sentence analysis requires disambiguating these different word forms to determine the correct semantic interpretation.

Slide 6: Ambiguity: pragmatics

Pragmatic ambiguity requires resolving references using context and world knowledge, which is trivial for humans but challenging for computers. This includes co-reference resolution, where a pronoun like "his" links to an antecedent such as "Joe," and identifying that phrases like "the first lady" refer to previously mentioned entities like "Jill." Additionally, temporal expressions like "five minutes later" depend on inferring the specific prior event they modify.

Slide 7: Difficulties

Machine translation faces significant challenges beyond simple word-for-word substitution, primarily due to differing word orders between languages. While English follows a modifier-noun structure (e.g., "a red house"), French uses noun-modifier order ("une maison rouge"). This reordering becomes increasingly complex in longer phrases like "almost double the usual oatmeal biscuits," where literal translation fails to produce grammatically correct French, necessitating sophisticated computational analysis.

Slide 8: Difficulties

Machine translation faces significant difficulties because words often lack straightforward one-to-one mappings across languages. Context determines the correct translation, as seen when English "leg" translates to distinct French terms like étape, jambe, or patte depending on usage. Additionally, idiomatic expressions such as "kick the bucket" require understanding figurative meaning rather than literal word substitution to achieve accurate results.

Slide 9: Difficulties

Machine translation faces significant difficulties because a single word can possess multiple distinct meanings. This ambiguity is illustrated by the French word "avocat," which requires context to determine its correct translation. Without resolving such polysemy, automated systems cannot accurately map source words to their intended target language equivalents.

Slide 10: Difficulties

Machine translation faces significant difficulties because a single word can possess multiple distinct meanings. This ambiguity is illustrated by the French word "avocat," which requires context to determine its correct translation. Without resolving such polysemy, automated systems cannot accurately map source text to target language equivalents.

Slide 11: Difficulties

Machine translation faces significant difficulties because words often possess multiple meanings, such as the French word "avocat," which translates to either "avocado" or "lawyer." Additionally, simple keyword matching fails in complex contexts; for instance, a system might incorrectly identify "Gary" as an explorer by matching isolated words like "May," "India," and "Portugal" without understanding the sentence's actual meaning.

Slide 12: Difficulties

Machine translation faces significant difficulties due to lexical ambiguity, where a single source word maps to multiple target meanings depending on context. The slide illustrates this with French terms like "pied" (foot/paw) and "jambe" (leg), which require disambiguation based on whether the subject is human or animal. Similarly, English words such as "chair" and "journey" demonstrate how semantic interpretation varies across different entities.

Slide 13: Difficulties

Machine translation faces difficulties because certain languages, like Spanish, omit subject pronouns and rely on verb endings (e.g., -o for "I", -a for "he/she/it") to indicate the subject. This creates ambiguity when translating to English, as a form like "habla" could mean "he," "she," or "it speaks." Consequently, systems must analyze context to determine the correct pronoun, as meaning overlaps and translation is not one-to-one.

Slide 14: Difficulties

Machine translation faces difficulties with pronoun resolution when sentences share identical superficial structures but require different semantic interpretations. In the example "The monkeys did not eat the bananas because they were not [hungry/ripe]," the pronoun "they" refers to monkeys in the first case and bananas in the second, necessitating world knowledge to resolve correctly. This ambiguity further complicates translation into French, where the chosen pronoun must match the grammatical gender of the specific antecedent.

Slide 15: Difficulties

This slide illustrates how machine translation is complicated by grammatical case differences between languages. While the English pronoun "her" remains constant across three sentences, its German translation changes to "sie," "ihr," or "ihrem" depending on whether it functions as an accusative direct object, a dative indirect object, or a possessive noun. These language-specific peculiarities require accurate functional analysis to select the correct word form.

Slide 16: Difficulties

The French phrase “je bois du lait” can translate as “I drink milk” or “I am drinking milk.” The choice depends on context because French present tense does not mark the same habitual-versus-progressive contrast as English.

Slide 17: Difficulties

“I drink milk” gives the habitual or general English reading of “je bois du lait.” It is not the only possible translation; the next build adds a progressive reading.

Slide 18: Difficulties

“I am drinking milk” gives a current-event reading. Comparing it with “I drink milk” shows why translation needs context rather than a fixed word-for-word mapping.

Slide 19: Difficulties

Idioms and proverbs often have conventional or figurative meanings that cannot be recovered by translating each word literally. A translation system must identify the expression and choose a target-language phrase that conveys its intended meaning.

Slide 20: Difficulties

Spell-checking faces difficulties because it must distinguish between homophones like "too" and "to" based on context rather than just verifying word existence. The examples demonstrate that determining whether a word functions as a comparison or an infinitive marker requires understanding the surrounding sentence structure to select the correct form.

Slide 21: Difficulties

Question answering takes a natural-language question and returns an answer grounded in relevant text or knowledge. This slide marks the application in the survey of language-processing difficulties.

Slide 22: Difficulties

This repeated Question Answering page is a transition marker. It identifies the task area but does not show a new example; the surrounding lecture uses it as one application affected by linguistic ambiguity.

Slide 23: Difficulties

Speech technology faces significant challenges in both recognition and synthesis due to linguistic ambiguities. Speech recognition struggles with homophones, illustrated by the "fish vs. ghoti" example and the phrase "The system is trained to wreck a nice peach." Conversely, speech synthesis must correctly distinguish between words with identical spellings but different pronunciations based on part of speech, such as the adjective versus noun forms of "content" or the verb versus noun forms of "address."

Slide 24: Difficulties

Speech technology faces significant difficulties due to linguistic ambiguities in both recognition and synthesis. In speech recognition, homophones like "recognize speech" and "wreck a nice peach" sound identical, confusing systems trained on specific phrases. Speech synthesis struggles with context-dependent pronunciation, where words like "content" and "address" shift stress patterns based on whether they function as nouns or verbs.

Slide 25: Difficulties

Speech technology faces difficulties due to ambiguity in both recognition and synthesis, as pronunciation and spelling lack a one-to-one mapping. The "fish vs. ghoti" example illustrates this by combining sounds from "rough," "women," and "nation" to spell a word that sounds like "fish." Additionally, systems struggle with homographs where stress determines meaning, such as distinguishing the adjective "conténт" from the noun "cóntent" or the verb "addréss" from the noun "áddress."

Slide 26: faculty of arts

This page is a section marker for the next topic. It signals a change in the lecture sequence; it does not introduce a new definition by itself.

Slide 27: Classification

Classification automatically assigns an instance, such as a sentence or text, to a predefined set of categories. Widely used in NLP, it includes document classification, sentiment analysis, part-of-speech tagging, and named entity recognition. While often binary (e.g., yes/no), tasks can also be multi-class or multi-label, requiring a labeled corpus of instances for machine learning to assign the correct class.

Slide 28: Rule-based

Rule-based sentiment analysis uses manually constructed symbolic rules to classify text, such as assigning a POSITIVE label if words like "exciting" appear or NEGATIVE if words like "hated" are present. While this approach is easy to interpret and adjust because the classification logic is explicit, it is labor-intensive to create and struggles to generalize across all possible expressions of sentiment.

Slide 29: Rule-based

Rule-based systems classify text by applying explicit conditional logic, such as outputting "SPORTS" if words like 'soccer' appear or "WEATHER" for terms like 'rainy'. While these methods are easy to interpret and adjust, they are labor-intensive to create and lack generalizability.

Slide 30: Supervised learning: statistics

Supervised learning employs a data-driven approach where the computer autonomously determines feature importance. The process takes as input a set of labels, such as {+, -}, and a collection of labeled documents like {(d1,+), (d2,-)}. The output is a trained classifier function F that maps documents to classes, raising key questions regarding the specific form of F and the methods used to learn it.

Slide 31: Supervised classifiers

A supervised classifier learns a mapping from labeled examples to one of a fixed set of classes. The slide lists Naive Bayes, logistic regression, support vector machines, and k-nearest neighbours as alternative classifier families.

Slide 32: Machine learning

Machine learning distinguishes between supervised and unsupervised approaches. Supervised learning trains a system on a classification task using a manually labeled training set to associate defined features with labels, with performance evaluated on a separate test set. Conversely, unsupervised learning automatically induces object representations from unlabeled data to generalize without manual intervention.

Slide 33: Supervised learning: feature-based learning

Supervised learning relies on features, which are elementary indices linking observed data to a predicted class. These features, derived from data such as word frequencies or Boolean presence, serve as the exclusive basis for prediction. Machine learning algorithms assign weights to these numerical, categorical, or boolean values to associate specific text characteristics with the target class.

Slide 34: Feature model

This slide illustrates feature models across three tasks: text categorization, author identification, and sentiment classification. In text categorization, the data is a document classified into categories like 'medical' using individual words as features represented by boolean presence or word counts. For author identification, such as classifying a book as written by Charles Dickens, the features consist of extracted n-grams from the document.

Slide 35: Feature-based model

The feature-based model employs a linear classifier to map a set of features {fi} to multiple classes {c}. Each feature is assigned a specific weight λi,c for every class, allowing features to vote based on these weights. The final classification selects the class c that maximizes the sum of these weighted features, with various optimization methods available for determining the optimal weights.

Slide 36: Feature-based model

This slide illustrates a linear classifier for text classification using lemma frequencies as features to distinguish between computer science and biology classes. For the sentence "The virus predominantly targets Windows computers," the model calculates a class score by summing the products of each word's frequency and its specific weight for that class. The resulting votes are 2.6 for computer science and -1.0 for biology, demonstrating how positive and negative weights determine the final prediction.

Slide 37: Feature-based model

Logistic regression transforms uninterpretable linear scores, which range from negative to positive infinity, into clear probabilities between 0 and 1. This is achieved by applying an exponential function to ensure non-negative values and then normalizing across all classes. Parameters are selected to maximize likelihood based on training data, converting raw scores like 2.6 and -1.0 into specific class probabilities such as 0.97 for computer science and 0.03 for biology.

Slide 38: Exponential function

The exponential function transforms input values ranging from negative infinity to positive infinity into output values strictly between zero and positive infinity. By mapping any real number to a positive range, this function effectively eliminates negative values. This transformation ensures that regardless of the input magnitude or sign, the resulting value is always non-negative.

Slide 39: Sentiment analysis

Sentiment analysis involves monitoring opinions regarding specific products, aggregating these individual viewpoints, and tracking resulting trends. This process enables the systematic evaluation of public sentiment within computational linguistics applications.

Slide 40: Sentiment analysis

Sentiment analysis involves monitoring public opinion regarding specific products, aggregating these individual opinions into collective insights, and tracking resulting trends over time. This process serves as a key application within computational linguistics for understanding consumer sentiment.

Slide 41: Sentiment analysis

Sentiment analysis involves monitoring opinions regarding specific products, aggregating these individual viewpoints, and tracking resulting trends over time. This process enables the systematic evaluation of public sentiment within computational linguistics contexts. The slide outlines these three core functions as the primary objectives for analyzing textual data to understand consumer feedback patterns.

Slide 42: Sentiment analysis

Sentiment analysis classifies text, such as movie reviews, into categories like positive or negative based on expressed opinions. While the simplest approach uses binary labels, more complex methods include adding a neutral category, assigning numerical scores from 1 to 5, or detecting sentiment toward specific aspects like actors versus direction. The process involves analyzing input sentences to assign the appropriate label reflecting the reviewer's stance.

Slide 43: Sentiment analysis

Sentiment analysis classifies text attitude across three complexity levels: simple (positive/negative), moderate (1-5 scale), and advanced (detecting targets, sources, and complex attitudes). Applications include movie reviews and election prediction, where models analyze individual words or combinations like bigrams to assign labels. While effective for standard cases, the professor notes that sarcasm and irony present significant challenges requiring more complex models.

Slide 44: Sentiment analysis

The base algorithm for sentiment analysis involves tokenization to identify words, followed by feature extraction to build a vocabulary. These features are then processed using classification methods such as Naive Bayes, logistic regression, or support vector machines. The lecture emphasizes using these techniques to automatically assign positive or negative classes to social media messages, with an immediate focus on explaining the Naive Bayes method.

Slide 45: Naive Bayes classification

Naive Bayes classification is a simple method grounded in Bayes' theorem. It operates by applying this probabilistic framework to documents represented using the bag-of-words model, which treats text as an unordered collection of words. This approach provides a foundational technique for computational linguistics tasks involving document categorization.

Slide 46: Bag of words for sentiment analysis

Slide 46 illustrates the Bag of Words approach for sentiment analysis using a positive review of a Bond movie. The example text, "This is one of the best Bond movies I have ever seen...", is processed to yield a specific count representation denoted as c 18/52. This notation reflects the resulting feature vector or token statistics derived from the input sentence within the computational linguistics framework.

Slide 47: Bag of words for sentiment analysis

Bag of words is a simple yet effective classification method for sentiment analysis using the Naive Bayes algorithm. The process involves selecting specific features from a vocabulary, such as adjectives like "best," "superbly," "interesting," and "great," which are identified as indicators of sentiment. These selected words serve as the basis for automatically determining the sentiment of input sentences.

Slide 48: Bag of words for sentiment analysis

The bag-of-words sentiment example represents a document by word counts and scores its positive terms. The displayed score is γ = 20/52 for this example; the representation ignores word order, so it can miss negation or phrase structure.

Slide 49: Bag of words for sentiment analysis

Slide 49 illustrates the Bag of Words representation for sentiment analysis using a specific feature vector example. The notation γ(best 1 superbly 1 interesting 1 twists 1 great 1) = c 21/52 demonstrates how individual word counts are aggregated to calculate a class probability or score, where the value 21/52 represents the computed result for this set of terms.

Slide 50: Bayes’ theorem for sentiment classification

Bayes' theorem for sentiment classification calculates the probability of a class c (positive or negative) given a document d. The formula is expressed as $P(c | d) = \frac{P(d | c)P(c)}{P(d)}$, where $P(d | c)$ represents the likelihood, $P(c)$ is the prior probability, and $P(d)$ is the evidence. This equation forms the mathematical basis for determining sentiment by combining these probabilistic components.

Slide 51: Bayes’ theorem for sentiment classification

Bayes' theorem for sentiment classification determines the most probable class c_{MAP} by maximizing the posterior probability $P(c|d)$. Since the document probability $P(d)$ is constant across classes, the optimization simplifies to maximizing the product of the likelihood $P(d|c)$ and the prior $P(c)$. The document d is further represented as a sequence of features $(x_1, x_2, \dots, x_n)$ within this likelihood term.

Slide 52: Bayes’ theorem for sentiment classification

Directly estimating the probability of a class given all features in a document is impossible without a gigantic training set. Bayes' theorem reverses this conditional probability, allowing us to compute the product of the prior class probability and the likelihood of the document given that class instead. The classification assigns the label corresponding to the maximum value of this product across candidate classes, as the denominator remains constant for a single document.

Slide 53: Independence hypotheses

To simplify probability estimation, two hypotheses are applied: the bag-of-words hypothesis assumes word order is irrelevant, while the conditional independence hypothesis assumes feature probabilities given a class are independent. This allows replacing the complex joint probability of all words with the product of individual word probabilities, expressed as $P(x_1, \dots, x_n | c) = P(x_1 | c) \cdot \dots \cdot P(x_n | c)$. Although this "naive" assumption ignores actual linguistic dependencies, it enables efficient computation and performs well in applications like sentiment classification.

Slide 54: Naive Bayes classifier

The Naive Bayes classifier determines the optimal class c by maximizing the posterior probability. While the Maximum A Posteriori (MAP) approach uses the joint likelihood $P(x_1, \dots, x_n | c)$, the Naive Bayes variant (c_{NB}) simplifies this by assuming feature independence, calculating the product of individual conditional probabilities $\prod P(x_i | c)$ multiplied by the class prior $P(c)$.

Slide 55: Train a naive Bayes classifier

To train a naive Bayes classifier using maximum likelihood estimation, probabilities are derived directly from data frequencies. The prior probability $P(c_j)$ is calculated as the count of documents in class c_j divided by the total number of documents. The conditional probability $P(w_i | c_j)$ is determined by the frequency of word w_i within class c_j, normalized by the sum of all word counts in that class.

Slide 56: Train a naive Bayes classifier

To train a Naive Bayes classifier, estimate the probability $P(w_i | c_j)$ by calculating the fraction of times word w_i appears among all words in documents of class c_j. Efficiently compute this by concatenating all documents of a specific class into a single "meta-document" and counting word frequencies within it. The professor further specifies applying smoothing by adding one to each word count and adding the vocabulary size to the total word count before applying the Naive Bayes formula for classification.

Slide 57: Problem with maximum likelihood estimation

Maximum likelihood estimation fails when a word like "fantastic" never appears in training documents for a specific class, resulting in a conditional probability of zero. Because Naive Bayes multiplies individual feature probabilities, this single zero destructively forces the entire product to zero, ignoring other positive evidence. To resolve this, smoothing is applied by adding one to both the numerator and denominator, ensuring no word receives a zero probability.

Slide 58: Laplace smoothing (add-1) for naive Bayes classification

Laplace smoothing (add-1) prevents zero probabilities in Naive Bayes by adding one to the count of every word in the vocabulary before normalization. This ensures that unseen words retain a slight non-zero probability, avoiding the destructive effect of multiplying by zero during classification. The adjusted formula calculates $P(w_i | c_j)$ as $(count(w_i, c_j) + 1)$ divided by the sum of all counts plus the vocabulary size $|V|$.

Slide 59: Naive bayes classifier: algorithm

The Naive Bayes algorithm trains by first extracting a vocabulary from the training corpus. It then computes the prior probability for each class by counting its document frequency relative to the total collection. Finally, it calculates the conditional probability of each word given a class by concatenating all documents of that class and applying Laplace smoothing to the word occurrence counts.

Slide 60: Other classifiers for sentiment classification

Naive Bayes is not the only option for sentiment classification; other approaches include logistic regression, support vector machines (SVM), and neural networks. Since robust implementations of these classifiers already exist, practitioners primarily need to define relevant features and provide training data rather than building algorithms from scratch. These methods associate feature-class combinations with weights to predict sentiment based on cues like specific words.

Slide 61: Sentiment classification: difficulties

Sentiment classification faces difficulties due to subtlety and differing expectations, which can mislead models like Naive Bayes. For instance, the phrase "should have been fantastic" implies a negative outcome despite the positive word "fantastic," while "disappointed to read negative comments" expresses a positive sentiment about other reviews rather than the current subject. These complexities, including irony and sarcasm, require more advanced approaches than simple keyword matching.

Slide 62: Structured prediction: sequences

Structured prediction for sequences involves many natural language processing tasks where the input is a sequence and the objective is to label each individual element. The slide illustrates this concept using the sentence "Charles Michel is the president of the European Council since december 2019" as an example of sequential data requiring element-wise labeling.

Slide 63: Structured prediction: sequences

Many natural language processing tasks involve structured prediction on sequences, where the objective is to label each element within the input sequence. A primary example is Part-of-Speech (POS) tagging, which assigns grammatical categories such as nouns, verbs, and adjectives to every word in a sentence like "Charles Michel is the president of the NPP."

Slide 64: Structured prediction: sequences

Many natural language processing tasks involve structured prediction on sequences, where the objective is to label each individual element. The slide illustrates this with named entity recognition, assigning specific tags like B-PER for persons, I-ORG for organizations, and O for non-entity tokens to words in a sentence. This process transforms raw sequential input into a structured output by categorizing every component according to its semantic role.

Slide 65: Part of speech

Part of speech categorizes words into classes that exhibit similar behaviors, providing essential context within a sentence. For instance, adjectives are often succeeded by nouns, while personal pronouns are frequently followed by verbs. This information is valuable for language processing applications, where the automatic assignment of these categories is known as part-of-speech tagging.

Slide 66: Named entity recognition

Named entity recognition is a critical subtask in natural language processing that involves finding and classifying names within text. The slide illustrates this concept using a sentence containing specific entities such as the organization "Eurostat," the date "Wednesday 28 October," and locations like "Paris" and "Berlin." These examples demonstrate how the task identifies distinct categories of named items embedded in a broader context.

Slide 67: Named entity recognition

Named entity recognition is an important subtask in natural language processing that involves finding and classifying names within a text. The slide illustrates this concept using a sentence about a Eurostat survey, where entities such as the organization "Eurostat," the date "Wednesday 28 October," and locations "Paris" and "Berlin" would be identified and categorized.

Slide 68: Named entity recognition

Named entity recognition is a key NLP subtask that identifies and classifies specific names within text. The slide illustrates this by extracting entities such as "Eurostat" (organization), "Wednesday 28 October" (date), and "Paris" and "Berlin" (locations) from a sample sentence about labor costs.

Slide 69: Named entity recognition

Named entity recognition is transformed into a sequence labeling task using the IOB encoding scheme, which labels tokens as Inside (I), Outside (O), or Beginning (B). This method specifically employs B-XX tags to distinguish between adjacent entities of the same type. For example, in the phrase "Wednesday 28 October," the date components are labeled sequentially as B-DAT followed by I-DAT tags to mark the entity's start and continuation.

Slide 70: Manual definition of features

Manual feature definition involves selecting specific attributes such as the current word, surrounding context words, character substrings, and word forms. It also incorporates additional linguistic annotations like Part-of-Speech (POS) tags and context labels for previous or next tokens. These features are extracted from text sequences to support computational linguistic analysis.

Slide 71: Manual definition of features

Manual feature definition involves selecting specific attributes such as the current word, surrounding context words, character substrings, and word forms. It also incorporates additional linguistic annotations like Part-of-Speech (POS) tags and context labels from previous or next tokens. These features are applied to text sequences, exemplified by the sentence "Paramedics treat woman and boy after crash in Stoke-on-Trent."

Slide 72: Manual definition of features

Manual feature definition involves specifying inputs such as the current word, surrounding context words, character substrings, and word forms. It also incorporates additional linguistic annotations like part-of-speech tags and context labels for previous or next tokens. These features are applied to sentences, exemplified by the text "Paramedics treat woman and boy after crash in Stoke-on-Trent."

Slide 73: Manual definition of features

Manual feature definition involves selecting specific attributes such as the current word, surrounding context words, character substrings, and word forms. It also incorporates additional linguistic annotations like Part-of-Speech (POS) tags and context labels from previous or next tokens. These features are extracted from text sequences to support computational linguistic analysis.

Slide 74: Manual definition of features

Manual feature definition involves selecting specific attributes such as the current word, surrounding context words, character substrings, and word forms. It also incorporates additional linguistic annotations like Part-of-Speech (POS) tags and context labels from previous or next tokens. These features are applied to text sequences, exemplified by the sentence "Paramedics treat woman and boy after crash in Xx-x-Xx."

Slide 75: Manual definition of features

Manual feature definition involves selecting specific attributes such as the current word, surrounding context words, character substrings, and word forms. It also incorporates additional linguistic annotations like Part-of-Speech (POS) tags and context labels for previous or next tokens. The slide illustrates these concepts using a sentence example where words are paired with their corresponding POS tags to demonstrate feature extraction.

Slide 76: Manual definition of features

Manual feature definition involves selecting specific attributes such as the current word, surrounding context words, character substrings, and word forms. It also incorporates additional linguistic annotations like Part-of-Speech (POS) tags and context labels for previous or next tokens. The slide illustrates this approach using a sentence example where words are paired with their corresponding POS tags and entity labels to demonstrate feature extraction.

Slide 77: Manual definition of features

Manual feature definition involves selecting specific attributes such as the current word, surrounding context words, character substrings, and word forms. It also incorporates additional linguistic annotations like Part-of-Speech (POS) tags and context labels for previous or next tokens. The slide illustrates these concepts using a sentence example where words are paired with their corresponding POS tags and entity labels.

Slide 78: Sequence tagging

Sequence tagging treats labeling a word as simple classification but incorporates predicted labels from previous words alongside observed features derived from current, preceding, and following context. This approach utilizes specific attributes like part-of-speech tags and morphological properties to determine the optimal label for the current token. The slide illustrates this using the phrase "crash in Stoke-on-Trent," where features such as hyphenation and surrounding POS tags inform the prediction of entity labels.

Slide 79: Global optimization

Global optimization addresses the problem where a series of local judgments fails to ensure global optimality. Instead of emitting a single answer based on these local conditions, the solution involves predicting the n best answers. This approach allows for the computation of the best global sequence rather than relying solely on potentially suboptimal local decisions.

Slide 80: Experiment

This slide introduces an experiment to determine which of three sentiment classifiers performs best: a symbolic list of hand-written rules, naive Bayes, and logistic regression. It identifies conducting experiments as the common procedure in Natural Language Processing (NLP) for comparing these distinct approaches. The core objective is to establish a method for evaluating and selecting the superior model among the symbolic and statistical options presented.

Slide 81: Train, development, test

Models must be evaluated on unseen data to ensure generalization rather than memorization. The training set estimates parameters, the development set guides hyperparameter choices like features and thresholds, and the test set is used only once at the end to report final performance. Typical splits are 80/10/10, though cross-validation may be applied to the training portion if data is scarce, while the test set remains strictly separate.

Slide 82: Leakage

Leakage occurs when information from the test set inadvertently enters the model, resulting in overly optimistic performance scores. This often happens through practices like selecting hyperparameters based on test results, computing vocabulary before data splitting, or including duplicate texts and same-author examples in both training and test sets. To prevent this, all learning and selection processes must be conducted without any access to the test set.

Slide 83: Baseline

A performance metric like 95% accuracy is meaningless without a baseline for comparison, as a trivial model predicting only the majority class can achieve this score while failing to detect minority cases. Baselines include simple strategies such as always predicting the most frequent label or using basic word lists. If a sophisticated model fails to outperform these simplest possible models, the result is considered negative.

Slide 84: Upper bound

The upper bound represents the best expected score, typically defined by human performance measured through inter-annotator agreement. This metric is calculated using the percentage of agreement or chance-corrected measures like Krippendorff's α between independent annotators, acknowledging that the gold standard's reliability depends on this consensus. While objective tasks like sentiment analysis may show 80-90% agreement, subjective tasks such as irony detection yield considerably lower bounds, providing a critical range for interpreting model scores.

Slide 85: Beyond accuracy

Slide 85 introduces the confusion matrix to evaluate classification beyond simple accuracy, defining True Positives (TP), False Positives (FP), False Negatives (FN), and True Negatives (TN). It presents formulas for Precision (P), Recall (R), and the F1 score. The slide illustrates that an "always-not toxic" baseline can achieve 95% accuracy while yielding 0% recall and F1 for the toxic class, demonstrating why accuracy alone is insufficient.

Slide 86: Multiple classes

For multi-class classification, compute Precision, Recall, and F1 per class before averaging. Micro-averaging pools all decisions, making the result dominated by frequent classes, while macro-averaging takes the mean of per-class scores so every class counts equally. These two averages can differ considerably when dealing with imbalanced classes.

Slide 87: Performance on sequences

Evaluating named entity recognition requires computing precision, recall, and F-measure over entities rather than individual words. An entity is considered correct only if both its boundaries and type match the test set; for instance, tagging "Michel" instead of "Charles Michel" or mislabeling "European Council" as LOC are errors. Per-word accuracy is misleading because most words carry an O label, inflating scores despite poor entity detection.

Slide 88: Thresholds

Naive Bayes and logistic regression output probabilities, requiring a decision threshold (e.g., classifying as toxic if $p > 0.5$) to make predictions. This threshold acts as a hyperparameter tuned on the development set, where lowering it increases recall but decreases precision, while raising it increases precision but reduces recall.

Slide 89: Significance

A small accuracy difference between models, such as 84.2% for Naive Bayes versus 84.7% for logistic regression, does not necessarily indicate superiority because test sets are samples and random components yield variable results. To ensure reliability, researchers should use sufficiently large test sets, repeat experiments with different seeds or folds to report means and standard deviations, and apply significance tests like McNemar's test or paired bootstrap to validate small differences.

Slide 90: Error analysis

While evaluation scores quantify how often a model makes mistakes, they do not explain why. Error analysis addresses this by inspecting a sample of errors from the development set, categorizing them to identify frequent patterns and actionable issues. This process often leverages linguistic knowledge with the specific goal of explaining underlying error patterns rather than just measuring their frequency.

Slide 91: Checklist

This checklist outlines eight essential practices for rigorous model evaluation: maintain a separate test set used only once, avoid data leakage, and compare results against a baseline. Researchers must select metrics appropriate for the task and class distribution, make all decisions including thresholds on the development set, and report result uncertainty. Finally, every claim should be supported by results, accompanied by a thorough error analysis.

Key points to remember

The lecture transitions to sentiment analysis and statistical modeling, introducing the Naive Bayes classifier built on the Bag-of-Words representation and conditional independence assumptions. It covers the mathematical derivation of Bayes' Theorem, the necessity of Laplace smoothing to handle zero probabilities, and the limitations of simple keyword matching when facing negation, sarcasm, or irony. Finally, the session outlines rigorous model evaluation protocols, emphasizing the separation of training, development, and test sets to prevent data leakage, the use of baselines and human upper bounds for context, and the preference for precision, recall, and F1 scores over accuracy in imbalanced or sequence-based tasks like Named Entity Recognition.